

RFC – Request for Change

Standard Operating Procedure (SOP)

1. What is RFC

RFC (Request for Change) is a structured mechanism used in feature-level demand forecasting to convert marketing feature selections into manufacturing demand quantities. RFC ensures accurate BOM explosion, planning, and supply-chain alignment.

Why RFC is Required

RFC bridges the gap between marketing intent (variants and options) and manufacturing execution (combinations and defaults). Without RFC, feature-based demand cannot be reliably translated into supply demand.

2. Feature Model Used in RFC

F-BOM (Feature Bill of Material) represents feature-level components used for forecasting.

Feature Type	Description
Variant (F)	Default marketing feature – one is always selected
Option (F)	Additional feature selected by customer
Combination (D)	Manufacturing grouping of multiple features
Default	Absence of feature (optional case)

3. RFC Logic Rules

Symbol	Meaning	Evaluation Rule
,	AND	Minimum value
OR	OR	Sum value
Absence of (X)	NOT X	$1 - X$

Script Run for RFC

1.Input Files preparation

Navigate to the server path:

- E:_Projects\RFC

Replace files in the input folder:

- E:_Projects\RFC\Input
 - Logic.xlsx – Excel format (DEP POR)
 - MCID Break.csv – UTF-8 CSV
 - Pivot.csv – UTF-8 CSV

Logic File Notes:

- Ensure 'Absence of' always contains parentheses
Example: Absence of (F332 OR F283 OR F289 OR F996)
- Maintain Maintain marketing and manufacturing in logic sheet and rest of the features like start from (OP, PS, VR, VF) are Maintain into PPL sheet
- Replace blanks with OR inside Absence expressions

Script running

Open vs code application and navigate : E:_Projects\RFC

Open file name ; rfc_automation.py

Out put : E:_Projects\RFC\Output

RFC Script Processing Flow

Load Inputs
Preprocess Logic
Split AND / OR
Extract Features
Compute Quantity
Update Pivot

Condition-wise Example

Expression: (A, B) OR (C)

A	B	C	min(A,B)	Final Qty
1	1	1	1	2
1	0	1	0	1
1	0	0	0	0

RFC Automation Script (Full Reference)

```
import pandas as pd
```

```
import numpy as np
```

```
import re
```

```
import os
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
#
```

```
# CONFIGURATION
```

```
#
```

```
PIVOT_CSV = r'C:\Users\sk385055\OneDrive - NCR ATLEOS\Global Ops  
Backup\RFC\Input\Pivot.csv'
```

```
LOGIC_XLSX = r'C:\Users\sk385055\OneDrive - NCR ATLEOS\Global Ops  
Backup\RFC\Input\Logic.xlsx'
```

```
OUTPUT_XLSX = r'C:\Users\sk385055\OneDrive - NCR ATLEOS\Global Ops  
Backup\RFC\Output\RFC.xlsx'
```

```
ITEM_PAT = r'\b(F\d{3}|OP\d{2}|D\d{3}|PS\d{2}|PS\d{3}|VR\d{2}|VF\d{2})\b'
```

```
CROSS_CLASS_PAT = re.compile(r'^\d{4}-.+')
```

```
def _needs_direct_eval(func_str):
```

```
    """
```

```
    Returns True for functions that can't be fully represented by Fun_Or/Fun_And columns:
```

- Absence of appears inside an AND chain (not at top level)
- OR followed by an AND-paren group: 'X OR (Y, Z)' where (Y,Z) has no OR keyword

"""

```
if not isinstance(func_str, str):
    return False

# Absence of at top level is fine; inside AND chain is not
top_abs = re.match(r'^Absence\s+of\s*\(', func_str, re.IGNORECASE)
if not top_abs and re.search(r'\s*Absence\s+of\s*\(', func_str, re.IGNORECASE):
    return True # Absence inside AND chain

# OR with AND-paren: " OR (" where inner has no OR keyword
for m in re.finditer(r'\bOR\s+\\([^\s]+\\)', func_str, re.IGNORECASE):
    if not re.search(r'\bOR\b', m.group(1), re.IGNORECASE):
        return True # AND-paren after OR

return False
```

#

LOAD

#

```
def load_inputs():
    print("[1/6] Loading inputs ...")
    pivot = pd.read_csv(PIVOT_CSV)
    pivot_bk = pivot.copy()
    xl = pd.read_excel(LOGIC_XLSX, sheet_name=None)
```

```
return pivot, pivot_bk, xl['PPL'].copy(), xl['Logic'].copy()
```

```
#
```

```
# PRE-PROCESS
```

```
#
```

```
def _preprocess_function(text):
```

```
    """
```

```
    Convert commas inside parentheses to ' OR ' only when:
```

```
    a) The paren group contains an explicit 'or'/'OR' keyword (OR-group)
```

```
    b) The paren group is part of an 'Absence of' expression (always SUM)
```

```
    Groups without 'or' keyword and not part of Absence-of are AND-groups (kept as-is).
```

```
    Condition 6 rows must be excluded entirely (caller's responsibility).
```

```
    """
```

```
    if not isinstance(text, str):
```

```
        return text
```

```
    # Step 1: Expand commas inside Absence-of parens to OR
```

```
    # "Absence of (A, B, C)" -> "Absence of (A OR B OR C)"
```

```
    def _abs_repl(m):
```

```
        return 'Absence of (' + re.sub(r'\s*,\s*', ' OR ', m.group(1)) + ')'
```

```
    text = re.sub(r'Absence\s+of\s*\V((\[^\])*)\V)', _abs_repl, text, flags=re.IGNORECASE)
```

```
    # Step 2: For remaining parens, only convert if they contain explicit OR keyword
```

```
    def _repl(m):
```

```

inner = m.group(1)

if re.search(r'\bor\b', inner, re.IGNORECASE):
    return '(' + re.sub(r'\s*,\s*', ' OR ', inner) + ')'

# AND-group: leave as-is
return m.group(0)

```

```

return re.sub(r'\s*([^\s]*)\s*', _repl, text)

```

```

def preprocess_df(df):
    """Apply pre-processing to all rows except Condition 6."""
    df = df.copy()
    mask = df['Condition'] != 6.0
    df.loc[mask, 'Function'] = df.loc[mask, 'Function'].apply(_preprocess_function)
    return df

```

```

#

```

```

# HELPERS: top-level splitting

```

```

#

```

```

def _split_or(expr):
    """Split by ' OR ' at top level (depth 0)."""
    parts, depth, cur, i = [], 0, "", 0
    while i < len(expr):
        ch = expr[i]
        if ch == '(': depth += 1; cur += ch; i += 1

```

```

elif ch == ')': depth -= 1; cur += ch; i += 1
elif depth == 0 and expr[i:i+4].upper() == ' OR ':
    parts.append(cur.strip()); cur = ""; i += 4
else:
    cur += ch; i += 1
parts.append(cur.strip())
return [p for p in parts if p]

```

```

def _split_comma(expr):
    """Split by comma at top level (depth 0)."""
    parts, depth, cur = [], 0, ""
    for ch in expr:
        if ch == '(': depth += 1; cur += ch
        elif ch == ')': depth -= 1; cur += ch
        elif ch == ',' and depth == 0:
            parts.append(cur.strip()); cur = ""
        else:
            cur += ch
    parts.append(cur.strip())
    return [p for p in parts if p]

```

#

FEATURE EXTRACTION

#

```
def extract_conditions(text):
```

```
    """
```

```
    Parse a (pre-processed) function string into:
```

```
    Fun_Or, Fun_Or_1, ... – items in parenthesised OR-groups (contain 'OR' keyword)
```

```
    Fun_And          – items in AND-groups (comma-only parens) + bare AND items
```

```
    After pre-processing only OR-paren-groups have been normalised; comma-only  
    paren groups remain unchanged and their items are added to Fun_And.
```

```
    """
```

```
    empty = {c: " for c in ['Fun_Or', 'Fun_And'] + [f'Fun_Or_{i}' for i in range(1, 6)]}
```

```
    if pd.isna(text) or str(text).strip() == ":
```

```
        return empty
```

```
    text = str(text).strip()
```

```
    result = {}
```

```
    remaining = text
```

```
    # 1. Parenthesised OR-groups (contain OR keyword)
```

```
    for i, m in enumerate(re.finditer(r'\s*([^\s\(\)]*\bOR\b[^\s\(\)]*\s*)\s*', remaining, re.IGNORECASE)):
```

```
        codes = re.findall(ITEM_PAT, m.group(1))
```

```
        col = 'Fun_Or' if i == 0 else f'Fun_Or_{i}'
```

```
        result[col] = f"[{', '.join(codes)}]" if codes else "
```

```
        remaining = remaining.replace(m.group(0), "", 1)
```

```
    # 2. Top-level OR (no parens – rare: "A OR B OR C" at top level)
```

```
    if 'Fun_Or' not in result:
```

```

top_or = [s.strip() for s in remaining.split(',')
           if re.search(r'\bOR\b', s, re.IGNORECASE)]

if top_or:
    codes = []
    for seg in top_or:
        codes += re.findall(ITEM_PAT, seg)
        remaining = remaining.replace(seg, ", 1")
    if codes:
        result['Fun_Or'] = f"[{','.join(codes)}]"

# 3. AND-paren-groups (no OR keyword inside) – extract items, add to Fun_And
# Skip parens that belong to "Absence of (...)" – handled in eval_standard
and_items = []
for m in re.finditer(r'\s*([^\s]*)\s*', remaining):
    inner = m.group(1)
    if re.search(r'\bOR\b', inner, re.IGNORECASE):
        continue # already handled as OR group
    # Check if this paren is part of "Absence of (...)"
    prefix_text = remaining[:m.start()].rstrip()
    if re.search(r'Absence\s+of\s*$', prefix_text, re.IGNORECASE):
        continue # leave Absence-of parens for eval_standard
    codes = re.findall(ITEM_PAT, inner)
    and_items.extend(codes)
    remaining = remaining.replace(m.group(0), ", 1")

# 4. Remaining bare AND items (comma-separated, no OR)

```

```

# Skip "Absence of ..." segments ONLY when they appear inside an AND chain
# (i.e., other AND items already exist). For top-level "Absence of (X)" functions,
# include the X items so Qty = sum(X), then Cond 1 applies 1 - Qty.

absence_is_entire = re.match(r'^\s*Absence\s+of\s*\(', text.strip(), re.IGNORECASE) and
not and_items

for seg in remaining.split(';'):

    seg = seg.strip()

    if not seg:

        continue

    is_absence = re.search(r'\bAbsence\s+of\b', seg, re.IGNORECASE)

    if is_absence:

        if absence_is_entire:

            # Extract items from the Absence of paren for SUM computation

            abs_m_bare = re.search(r'Absence\s+of\s*\([^\)]+\)', seg, re.IGNORECASE)

            if abs_m_bare:

                abs_codes = re.findall(ITEM_PAT, abs_m_bare.group(1))

                for code in abs_codes:

                    if code not in and_items:

                        and_items.append(code)

            else:

                pass # Absence inside AND chain: handled by eval_standard via direct eval

                continue

        if not re.search(r'\bOR\b', seg, re.IGNORECASE):

            m2 = re.search(ITEM_PAT, seg)

            if m2 and m2.group() not in and_items:

                and_items.append(m2.group())

```

```
result['Fun_And'] = f"[{'', '.join(and_items)}]" if and_items else "
```

```
for col in ['Fun_Or', 'Fun_And'] + [f'Fun_Or_{i}' for i in range(1, 6)]:
```

```
    result.setdefault(col, "")
```

```
return result
```

```
def apply_extraction(df):
```

```
    cond_cols = df['Function'].apply(
```

```
        lambda x: pd.Series(extract_conditions(x)) if isinstance(x, str) else pd.Series()
    ).fillna("")
```

```
    df = pd.concat([df, cond_cols], axis=1)
```

```
    df = df.loc[:, ~(df.apply(lambda c: c.astype(str).str.strip().eq(").all()))]
```

```
    return df
```

```
def prefix_fun_cols(df):
```

```
    """
```

```
    Prepend 4-char item class to every code in Fun_* columns.
```

```
    Codes already containing a CLASS-ITEM pattern (e.g. '5845-F001') are kept as-is.
```

```
    """
```

```
    fun_cols = [c for c in df.columns if c.startswith('Fun_')]
```

```
    cross_cls = re.compile(r'^\d{4}-.+')
```

```
def _pfx(item, fun_str):
```

```
    if not isinstance(fun_str, str) or not fun_str.strip():
```

```
        return []
```

```

pfx = str(item)[:4]

codes = [c.strip() for c in fun_str.strip('[]').split(',') if c.strip()]

return [c if cross_cls.match(c) else f"{pfx}-{c}" for c in codes]

```

```

for col in fun_cols:

    df[col] = df.apply(lambda r: _pfx(r['Item'], r[col]), axis=1)

return df

```

```

#

```

```

# ITEM LOOKUP HELPER

```

```

#

```

```

def _get(code, prefix, pv):
    """
    Resolve a single item code against the product's working-pivot dict.
    If code already has its own class prefix (e.g. '5845-F001') use it directly.
    Falls back to uppercase suffix for case-insensitive matching.
    """
    if re.match(r'^\d{4}-.+', code):
        key = code
    else:
        key = f"{prefix}-{code}"
    v = pv.get(key, pv.get(f"{prefix}-{code.upper()}", 0))
    return float(v) if pd.notna(v) else 0.0

```

#

CONDITION 6 EVALUATOR (OR-of-AND-groups)

#

```
def _eval_and_group(inner, prefix, pv):
```

```
    """
```

```
    Evaluate items inside a paren AND-group (comma-separated AND semantics).
```

```
    Handles Absence-of terms anywhere in the group.
```

```
    Returns MIN of all term values.
```

```
    """
```

```
    terms = _split_comma(inner)
```

```
    vals = []
```

```
    for term in terms:
```

```
        term = term.strip()
```

```
        abs_m = re.match(r'^Absence\s+of\s*\V{([\^]+)\V}$!', term, re.IGNORECASE)
```

```
        if abs_m:
```

```
            items = re.findall(ITEM_PAT, abs_m.group(1))
```

```
            vals.append(1.0 - sum(_get(c, prefix, pv) for c in items))
```

```
        else:
```

```
            items = re.findall(ITEM_PAT, term)
```

```
            if items:
```

```
                vals.append(_get(items[0], prefix, pv))
```

```
    return min(vals) if vals else 0.0
```

```
def eval_cond6(func_str, prefix, pv):
```

```
"""
```

Condition 6: (A,B) OR (C,D,Absence of(X)) OR (E,F)

= SUM(MIN(A,B), MIN(C,D,1-X), MIN(E,F))

Falls back to standard AND/OR evaluation when no top-level OR groups found.

```
"""
```

```
func_str = str(func_str).strip()
```

```
or_groups = _split_or(func_str)
```

```
if len(or_groups) > 1:
```

```
    total = 0.0
```

```
    for grp in or_groups:
```

```
        grp = grp.strip()
```

```
        if grp.startswith('(') and grp.endswith(')'):
```

```
            total += _eval_and_group(grp[1:-1], prefix, pv)
```

```
        else:
```

```
            # Single item outside parens
```

```
            items = re.findall(ITEM_PAT, grp)
```

```
            total += _get(items[0], prefix, pv) if items else 0.0
```

```
    return total
```

```
# No top-level OR – fall back to standard AND/OR
```

```
return eval_standard(func_str, prefix, pv)
```

```
#
```

```
# STANDARD AND/OR EVALUATOR (Conditions 3/4.5/5)
```

#

```
def eval_standard(func_str, prefix, pv):
```

```
    """
```

```
    Standard AND/OR evaluation.
```

```
    Handles:
```

- Absence of (items) = 1 - SUM(items) at top level or as AND term
- comma at top level = AND = MIN
- OR inside parens = SUM (after pre-processing)
- AND-only parens = MIN (commas only, no OR keyword)
- Cross-class refs = looked up as-is (e.g. '5845-F001')

```
    """
```

```
    func_str = str(func_str).strip()
```

```
    # Top-level Absence of
```

```
    abs_m = re.match(r'^Absence\s+of\s*\V(?:[^\s]+)\$', func_str, re.IGNORECASE)
```

```
    if abs_m:
```

```
        items = re.findall(ITEM_PAT, abs_m.group(1))
```

```
        return 1.0 - sum(_get(c, prefix, pv) for c in items)
```

```
    # AND (comma at top level) = MIN
```

```
    and_parts = _split_comma(func_str)
```

```
    if len(and_parts) > 1:
```

```
        vals = []
```

```
        for part in and_parts:
```

```

part = part.strip()

# Absence of as AND term
abs_m2 = re.match(r'^Absence\s+of\s*\(([^\)]+)\)$', part, re.IGNORECASE)

if abs_m2:
    items = re.findall(ITEM_PAT, abs_m2.group(1))
    vals.append(1.0 - sum(_get(c, prefix, pv) for c in items))
elif part.startswith('(') and part.endswith(')'):
    # Recursively evaluate the inner expression
    vals.append(eval_standard(part[1:-1], prefix, pv))
else:
    # Could have cross-class ref like '5845-F001'
    cross = re.findall(r'\d{4}-[A-Za-z0-9]+', part)
    items = re.findall(ITEM_PAT, part)

    if cross and not items:
        vals.append(_get(cross[0], prefix, pv))
    elif items:
        vals.append(_get(items[0], prefix, pv))

return min(vals) if vals else 0.0

# OR at top level = SUM; each part could be AND-paren, OR-paren, or single item
or_parts = _split_or(func_str)

if len(or_parts) > 1:
    total = 0.0

    for part in or_parts:
        part = part.strip()

        if part.startswith('(') and part.endswith(')'):

```

```

inner = part[1:-1]

if re.search(r'\bOR\b', inner, re.IGNORECASE):
    total += sum(_get(c, prefix, pv) for c in re.findall(ITEM_PAT, inner))
else:
    # AND-paren group: MIN of items
    and_items = [s.strip() for s in inner.split(',') if s.strip()]
    sub_vals = []
    for ai in and_items:
        cross = re.findall(r'\d{4}-[A-Za-z0-9]+', ai)
        items2 = re.findall(ITEM_PAT, ai)
        if cross:
            sub_vals.append(_get(cross[0], prefix, pv))
        elif items2:
            sub_vals.append(_get(items2[0], prefix, pv))
    total += min(sub_vals) if sub_vals else 0.0
else:
    items = re.findall(ITEM_PAT, part)
    if items:
        total += _get(items[0], prefix, pv)
return total

items = re.findall(ITEM_PAT, func_str.strip('()'))
return _get(items[0], prefix, pv) if items else 0.0

```

#

```
# QTY COMPUTATION
```

```
#
```

```
def compute_qty(data_fin, working_pivot, is_cond6=False):
```

```
    df1 = data_fin.copy()
```

```
    pv_dict = working_pivot.set_index('BOM Parent Product ID').to_dict(orient='index')
```

```
    if is_cond6:
```

```
        def _c6(row):
```

```
            pv = pv_dict.get(row['BOM Parent Product ID'], {})
```

```
            return eval_cond6(row.get('Function_orig', ''), str(row['Item'][:4], pv)
```

```
        df1['Qty'] = df1.apply(_c6, axis=1)
```

```
        df1['Qty'] = pd.to_numeric(df1['Qty'], errors='coerce').fillna(0)
```

```
    return df1
```

```
# Standard path
```

```
fun_or_cols = [c for c in df1.columns if c.startswith('Fun_Or') and not c.endswith('_Qty')]
```

```
fun_or_qty_cols = []
```

```
for i, col in enumerate(fun_or_cols):
```

```
    qty_col = 'Fun_Or_Qty' if i == 0 else f'Fun_Or_Qty_{i}'
```

```
    fun_or_qty_cols.append(qty_col)
```

```
def _sum_or(row, col=col):
```

```
    items = row[col]
```

```

    if not isinstance(items, list) or not items:

        return np.nan

    pv = pv_dict.get(row['BOM Parent Product ID'], {})

    return sum(float(pv.get(it, pv.get(it.upper(), 0)) or 0) for it in items)

df1[qty_col] = df1.apply(_sum_or, axis=1)

def _min_and(row):

    items = row.get('Fun_And', [])

    if not isinstance(items, list) or not items:

        return np.nan

    pv = pv_dict.get(row['BOM Parent Product ID'], {})

    return min(float(pv.get(it, pv.get(it.upper(), 0)) or 0) for it in items)

df1['Fun_And_Qty'] = df1.apply(_min_and, axis=1)

for i, col in enumerate(fun_or_cols):

    df1.loc[df1[col].apply(lambda x: not isinstance(x, list) or not x),

        fun_or_qty_cols[i]] = np.nan

df1.loc[df1['Fun_And'].apply(lambda x: not isinstance(x, list) or not x),

        'Fun_And_Qty'] = np.nan

qty_idx = df1.columns.get_loc(fun_or_qty_cols[0]) if fun_or_qty_cols else None

def _min_all(row):

    if qty_idx is None:

```

```
    return 0

    vals = row.iloc[qty_idx:]

    return 0 if vals.isnull().all() else vals.min()
```

```
df1['Qty'] = df1.apply(_min_all, axis=1)

df1['Qty'] = pd.to_numeric(df1['Qty'], errors='coerce').fillna(0)
```

```
mask1 = df1['Condition'].isin([1.0, 1.1])

df1.loc[mask1, 'Qty'] = 1 - df1.loc[mask1, 'Qty']

df1.loc[df1['Condition'] == 2.0, 'Qty'] = 0

df1['Qty'] = pd.to_numeric(df1['Qty'], errors='coerce').fillna(0)
```

```
# Override with direct eval for rows that need special handling

if 'needs_direct_eval' in df1.columns and 'Function_orig' in df1.columns:
```

```
    direct_mask = df1['needs_direct_eval'] == True
```

```
    if direct_mask.any():
```

```
        def _direct_eval(row):
```

```
            pv = pv_dict.get(row['BOM Parent Product ID'], {})
```

```
            val = eval_standard(row['Function_orig'], str(row['Item']))[:4], pv)
```

```
            cond = float(row['Condition']) if pd.notna(row['Condition']) else 3.0
```

```
            if cond in (1.0, 1.1):
```

```
                return 1.0 - val
```

```
            elif cond == 2.0:
```

```
                return 0.0
```

```
            return val
```

```
    df1.loc[direct_mask, 'Qty'] = df1[direct_mask].apply(_direct_eval, axis=1)
```

```
df1['Qty'] = pd.to_numeric(df1['Qty'], errors='coerce').fillna(0)
return df1
```

```
#
```

```
# PHASE RUNNER
```

```
#
```

```
def run_phase(sheet_df, cond_filter, working_pivot, label, is_cond6=False):
```

```
    print(f" {label} ...", end=' ', flush=True)
```

```
    sub = sheet_df[sheet_df['Condition'].isin(cond_filter)].copy()
```

```
    sub = sub[sub['Function'].notna() & sub['Function'].str.strip().ne("")]
```

```
    if sub.empty:
```

```
        print("(no rows)")
```

```
    return pd.DataFrame(), working_pivot
```

```
    sub['Function_orig'] = sub['Function'].copy()
```

```
    sub = preprocess_df(sub)
```

```
    sub['Class'] = sub['Item'].str[:4]
```

```
    if not is_cond6:
```

```
        sub = apply_extraction(sub)
```

```

sub = prefix_fun_cols(sub)

pv_class = working_pivot[['Class', 'BOM Parent Product ID']].drop_duplicates()
data_fin = pd.merge(pv_class, sub, on='Class', how='inner')
data_fin = data_fin[data_fin['Item'].notna()]

# Flag rows that need direct evaluation (can't be handled by Fun_Or/Fun_And split)
if not is_cond6 and 'Function_orig' in data_fin.columns:
    data_fin['needs_direct_eval'] = data_fin['Function_orig'].apply(_needs_direct_eval)

drop_these = ['Class', 'Function'] if is_cond6 else ['Class', 'Function']
data_fin.drop(columns=[c for c in drop_these if c in data_fin.columns],
              inplace=True, errors='ignore')

result = compute_qty(data_fin, working_pivot, is_cond6=is_cond6)
result.drop_duplicates(subset=['BOM Parent Product ID', 'Item'], inplace=True)
print(f"({len(result):,} rows)")

piv_new = result.pivot_table(
    index='BOM Parent Product ID', columns='Item', values='Qty', fill_value=0
).reset_index()

updated = pd.merge(working_pivot, piv_new, on='BOM Parent Product ID',
                  how='left').fillna(0)

return result, updated

#

```

```
# MAIN
```

```
#
```

```
def main():
```

```
    pivot, pivot_bk, ppl_raw, logic_raw = load_inputs()
```

```
    wp = pivot.copy()
```

```
    wp['Class'] = wp['BOM Parent Product ID'].str[:4]
```

```
    print("[2/6] Running computation phases ...")
```

```
    res5, wp = run_phase(logic_raw, [5.0], wp, "Phase 1 | Logic Cond 5 (PS/VR/OP)")
```

```
    res6, wp = run_phase(logic_raw, [6.0], wp, "Phase 2 | Logic Cond 6 (OR-of-AND)",  
is_cond6=True)
```

```
    res_ppl, wp = run_phase(ppl_raw, [3.0, 4.5], wp, "Phase 3 | PPL Cond 3/4.5")
```

```
    res_log, wp = run_phase(logic_raw, [1.0, 1.1, 2.0, 3.0, 4.5], wp, "Phase 4 | Logic Cond  
1/2/3/4.5")
```

```
    print("[3/6] Combining results ...")
```

```
    all_res = pd.concat([r for r in [res5, res6, res_ppl, res_log] if not r.empty],
```

```
                        ignore_index=True)
```

```
    all_res['Class'] = all_res['Item'].str[:4]
```

```
    all_res.drop_duplicates(subset=['BOM Parent Product ID', 'Item'], inplace=True)
```

```
    print("[4/6] Attaching logic functions ...")
```

```
    all_logic = pd.concat([logic_raw, ppl_raw], ignore_index=True)
```

```
    final = pd.merge(
```

```

all_res[['Class', 'BOM Parent Product ID', 'Item', 'Condition', 'Qty']],
all_logic[['Item', 'Function']], on='Item', how='left'
)[['Class', 'BOM Parent Product ID', 'Item', 'Function', 'Condition', 'Qty']]
final.drop_duplicates(subset=['BOM Parent Product ID', 'Item'], inplace=True)

print("[5/6] Building RFC pivot ...")
rfc_piv = all_res.pivot_table(
    index='BOM Parent Product ID', columns='Item', values='Qty', fill_value=0
).reset_index()

# Build Updated_Pivot: start from original pivot, update/add RFC computed columns
updated_pivot = pivot_bk.copy()
for col in rfc_piv.columns:
    if col == 'BOM Parent Product ID':
        continue
    # Merge this column (update existing or add new)
    col_data = rfc_piv[['BOM Parent Product ID', col]]
    if col in updated_pivot.columns:
        # Update existing column with RFC value
        updated_pivot = updated_pivot.drop(columns=[col])
    updated_pivot = pd.merge(updated_pivot, col_data, on='BOM Parent Product ID',
how='left')
    updated_pivot = updated_pivot.fillna(0)

print(f"[6/6] Writing {OUTPUT_XLSX} ...")
os.makedirs(os.path.dirname(os.path.abspath(OUTPUT_XLSX)) or '.', exist_ok=True)

```

```

keep = ['BOM Parent Product ID', 'Item', 'Condition', 'Qty',
        'Fun_Or', 'Fun_And', 'Fun_Or_Qty', 'Fun_And_Qty']

split_df = all_res[[c for c in keep if c in all_res.columns]].copy()

with pd.ExcelWriter(OUTPUT_XLSX, engine='xlsxwriter') as writer:
    final.to_excel(writer,    sheet_name='RFC',    index=False)
    split_df.to_excel(writer,  sheet_name='Logic_Splitup',index=False)
    updated_pivot.to_excel(writer, sheet_name='Updated_Pivot',index=False)

print(f"\n Done! -> {OUTPUT_XLSX}")

print(f" RFC      : {len(final):,} rows | {final['Item'].nunique():,} unique items")
print(f" Logic_Splitup: {len(split_df):,} rows")
print(f" Updated_Pivot: {len(updated_pivot):,} rows x {len(updated_pivot.columns):,}
cols")

if __name__ == '__main__':
    main()

```